

Chapter-3

Role of Lexical Analyzer: The lexical analyzer is the 1st phase of a compiler. Its main task is to read the input characters and produce output as a sequence of tokens that the parser uses for syntax analysis.

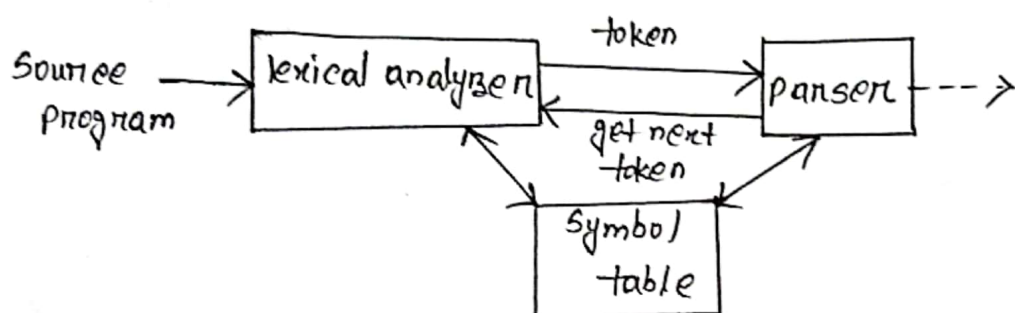


Fig: Interaction of lexical analyzer with parser

lexical analyzers are divided into a cascade of two phases, the 1st called scanning and 2nd called lexical analysis. The scanner is responsible for doing simple tasks. While the lexical analyzer proper does the more complex operations.

Issues in lexical analysis:

- ① Simpler design is perhaps the most important consideration. Separation of lexical analysis from syntax analysis often allows us to simplify one or the other three phases.
- ② Compiler efficiency is improved.
- ③ Compiler portability is enhanced.

Input Buffer: Input buffering is a technique used in compiler to read input characters from a source file and store them in a buffer before parsing them. The buffer can be of fixed or variable size. It helps reducing the number of input operations.

→ we introduce some useful techniques for speeding up the lexical analyzer. Such as sentinels.

There are three general approaches to the implementation,

- ① Use a lexical analyzer such as lex compiler.
- ② Write the lexical analyzer in a conventional systems programming language.
- ③ Write the lexical analyzer in assembly language.

Regular expression: The language accepted by finite automata can be described by simple expression called regular expression. It is a method to represent regular expression language.

- Operations:
- ① Union ($A \cup B$ or $A + B$)
 - ② Concatenation ($A \circ B$ or AB)
 - ③ Star (A^*); $*$ means 0 to many occurrences.
 - ④ Plus (A^+); $+$ means 1 to many occurrences.

Regular language to Regular expression:

- ① No string $\rightarrow \emptyset$ or $\{\}$
- ② Length 0 $\rightarrow \{\epsilon\}$ [Empty string]
- ③ Length 2 = $\{aa, ab, ba, bb\}$
 $= (aa + ab + ba + bb)$
 $= a(a + b) + b(a + b)$

$$= (a+b)(a+b)$$

$$\text{Length 3} = (a+b)(a+b)(a+b)$$

$$(4) \text{ At most 1 } \rightarrow \{ \epsilon, ab \} = \{ \epsilon + a+b \}$$

$$(5) \text{ At most 2 } \rightarrow \{ \epsilon + a+b \} \{ \epsilon + a+b \}$$

$$(6) \text{ Not more than 2 b's and 1 a} \\ = \{ \epsilon, b, a, ab, ba, bb, abb, bab, bba \}$$

Example: (1) All strings having a single 'b' over $\Sigma = \{a, b\}$
 $\Rightarrow a^* b a^*$

(2) All strings having at least one 'b'
 $\Rightarrow (a+b)^* b (a+b)^*$

(3) All strings having 'bbbb' as substring
 $\Rightarrow (a+b)^* bbbb (a+b)^*$

(4) All strings ends with 'ab'
 $\Rightarrow (a+b)^* ab$

(5) All strings containing 'a'
 $\Rightarrow (a+b)^* a (a+b)^*$

(6) All strings start and ends with different symbol

$\Rightarrow \text{Case 1} \Rightarrow a(a+b)^* b$

$\text{Case 2} \Rightarrow b(a+b)^* a$

(7) All strings start and end with 'a'
 $\Rightarrow a(a+b)^* a$

(8) All strings having 2 b's exact
 $\Rightarrow a^* b a^* b a^*$

(9) Every 0's followed by at least one '1'
 $\Rightarrow 1^* (01)^*$

Regular expression to regular language:

① $0^* 1 0^*$

$\Rightarrow \{w \mid w \text{ contains only one '1'}\}$

② $(0+1)^* 1 (0+1)^*$

$\Rightarrow \{w \mid w \text{ contains at least one '1'}\}$

③ $((a+b)(a+b))^*$

$\Rightarrow \{w \mid w \text{ has even length}\}$

④ $(a+b)^*$

$\Rightarrow \{w \mid w \text{ set of strings a's and b's of any length}\}$

⑤ $(a+b)^* abb$

$\Rightarrow \{w \mid w \text{ set of strings of a's and b's ending with the string abb}\}$

⑥ $(11)^*$

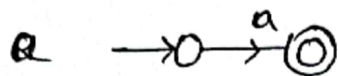
$\Rightarrow \{w \mid w \text{ even number of '1's'}\}$

⑦ $(aa)^* (bb)^* b \Rightarrow \{w \mid w \text{ even number of a's followed by odd number b's}\}$

⑧ $(aa+ab+ba+bb)^* \Rightarrow \{w \mid w \text{ string of a's and b's of even length including null}\}$

Convert Regular expression to NFA:

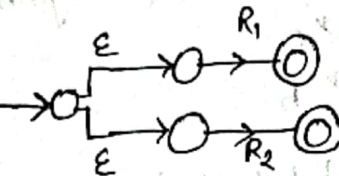
Step 1, $R = a$ where $a \in \Sigma$ (alphabets)



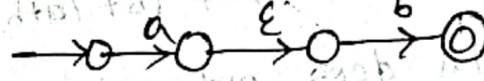
Steps ② $R = \epsilon$ (empty string) , $\rightarrow \odot$

③ $R = \emptyset$ (empty), $\rightarrow \emptyset$

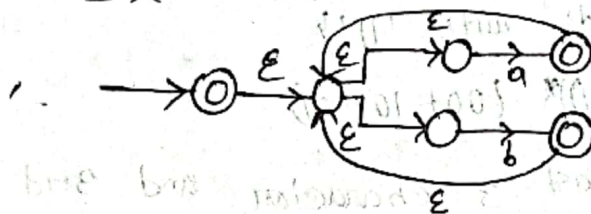
(4) $R = [R_1 \cup R_2] \cap [R_1 \cap R_2]$



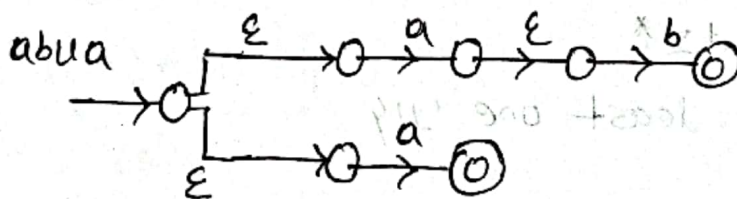
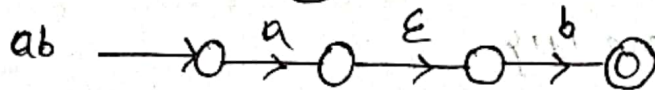
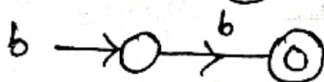
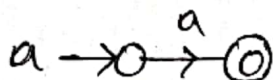
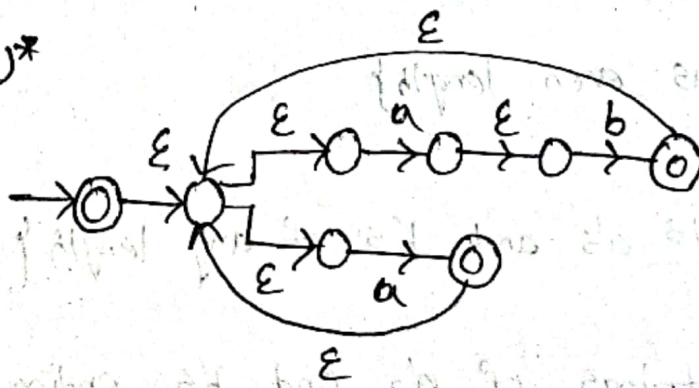
⑤ $R = [R_1, R_2] [R_1, R_2]$ (concentration)



⑥ (aib)*

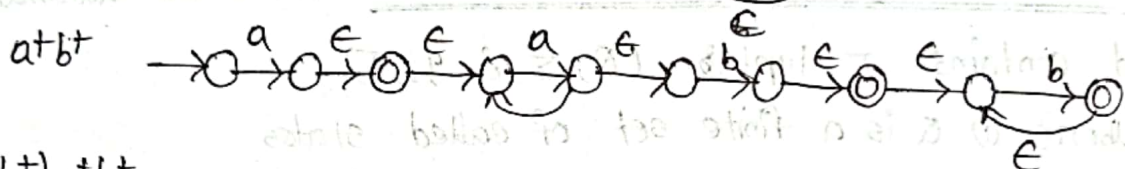
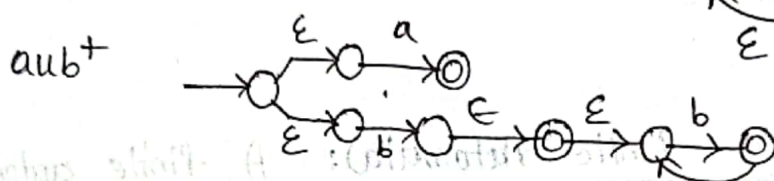
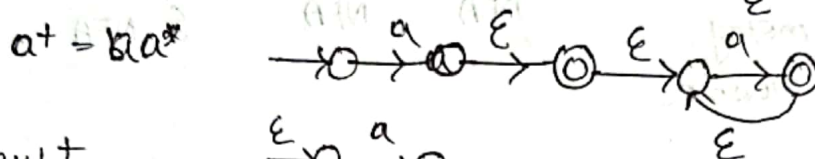
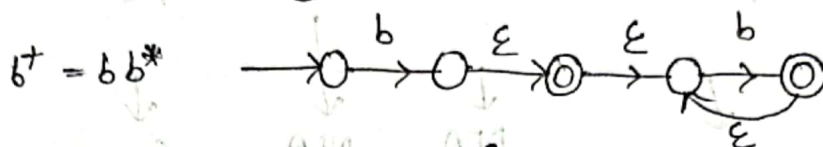
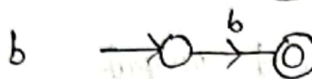
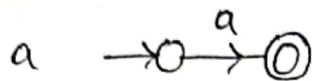


Example: ① $(abv)^*$

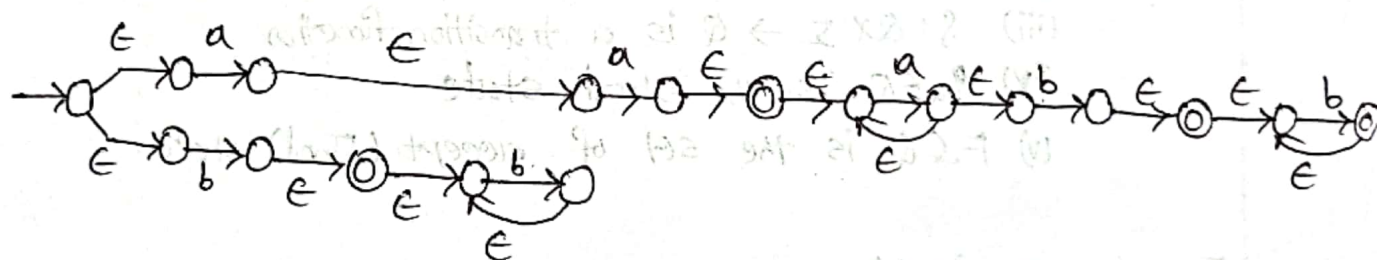

$$(abva)^*$$


② $(a+ b^+) a^+ b^+$

Solution:



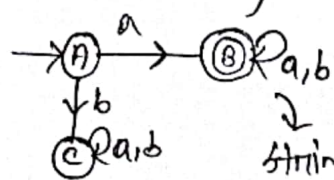
$(a+ b^+) a^+ b^+$



Finite automata: when we compile a regular expression into a recognizer by constructing a generalized transition diagram called a finite automaton. Both deterministic and non deterministic finite automata are capable of it.

$L(H) =$ set of all strings of length 2
 $= \{aa, ab, ba, bb\}$ - Finite set

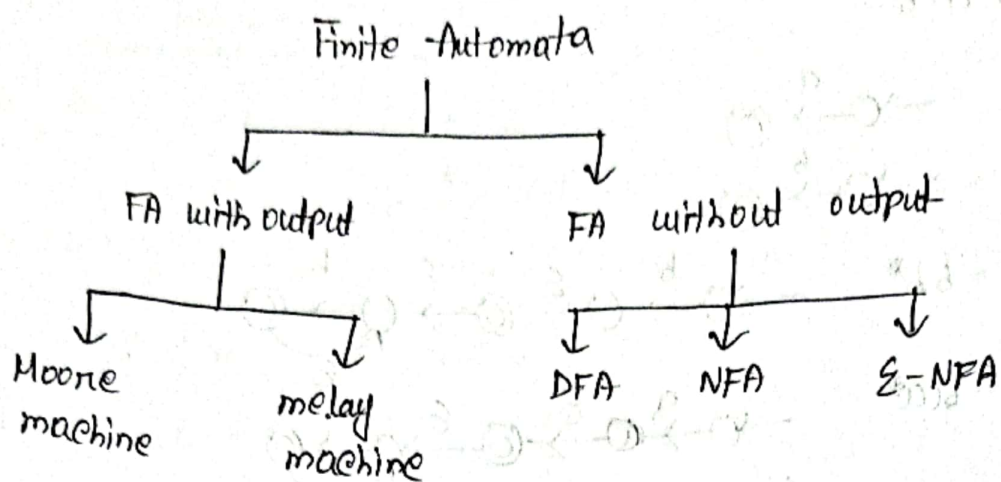
$L(H) =$ set of all strings start with 'a'
 $= \{a, ab, aab, \dots\}$ - Infinite set



String = $\{a, ab\}$ - accepted

String = $\{baa\}$ - Rejected

Finite Automata



DFA (Deterministic Finite Automata): A finite automaton

that contains 5-tuples: $(Q, \Sigma, \delta, q_0, F)$

where (i) Q is a finite set of called states

(ii) Σ is a finite set called alphabets

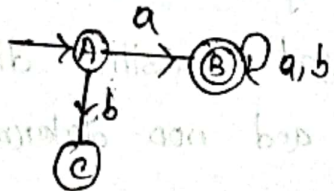
(iii) $\delta: Q \times \Sigma \rightarrow Q$ is a transition function

(iv) $q_0 \in Q$ is the start state

(v) $F \subseteq Q$ is the set of accept (Final) state.

Example: $\Sigma = \{a, b\}$

$L(M) =$ set of all strings that start with an 'a'



(i) $Q =$ set of all states

$Q = \{A, B, C\}$

(ii) $\Sigma =$ input alphabet

$= \{a, b\}$

(iii) $\delta = Q \times \Sigma \rightarrow Q$

	a	b
$\rightarrow A$	B	C
*B	B	B
C	C	C

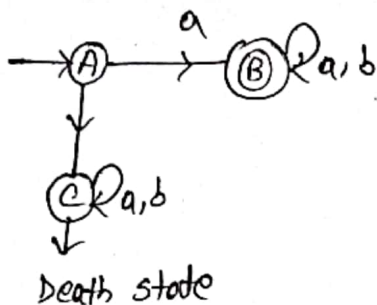
(iv) q_0 - start state = A

(v) F = set of final states
 $= \{B\}$

DFA construction:

① $L(M) = \{w \mid w \text{ accept set of all strings each strings start with an } a\}$

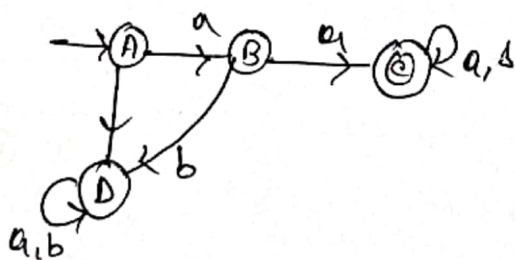
$\Sigma = \{a, b\}$



② $L(M) = \{w \mid w \text{ starts with an 'aa'}\}$

$\Sigma = \{a, b\}$

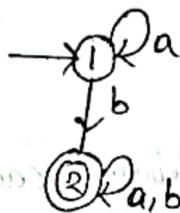
$= \{aa, aab, aaa, aabbaa, \dots\}$



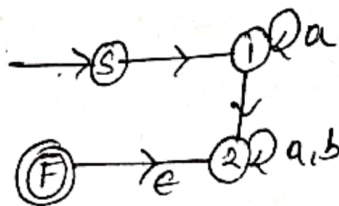
DFA to Regular expression:

- ① Convert the DFA to GNFA by adding a start state with an ϵ arrow with old start and a new accept state with an ϵ arrow with old accept and make previous final states to normal state.
- ② Remove the middle order states by ascending order.

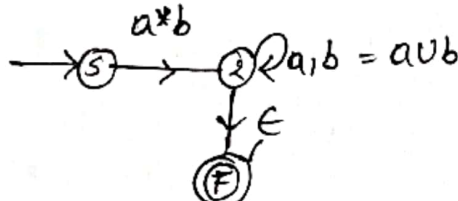
Example-1



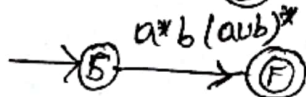
Solution: Step-1



Step-2

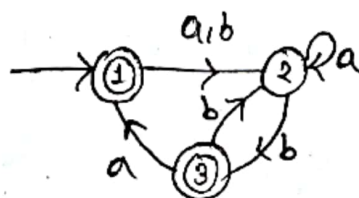


Step-3

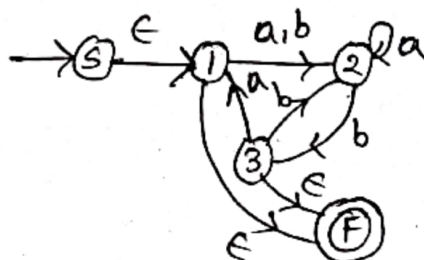


Regular expression $a^*b(aub)^*$

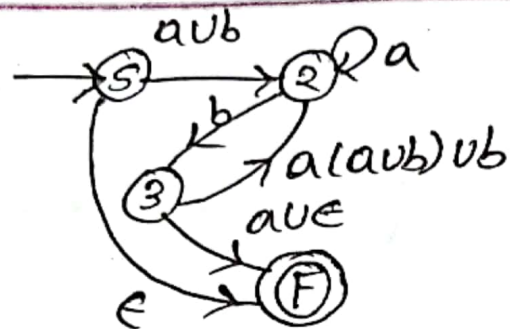
Example-2



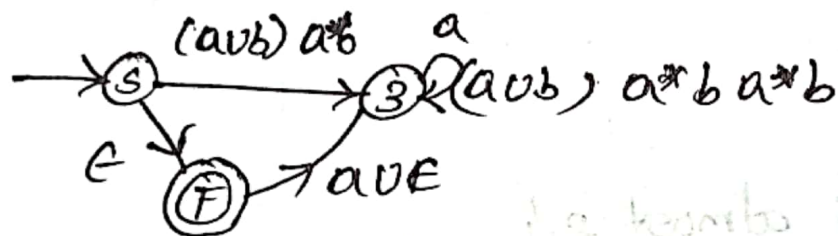
Step-1



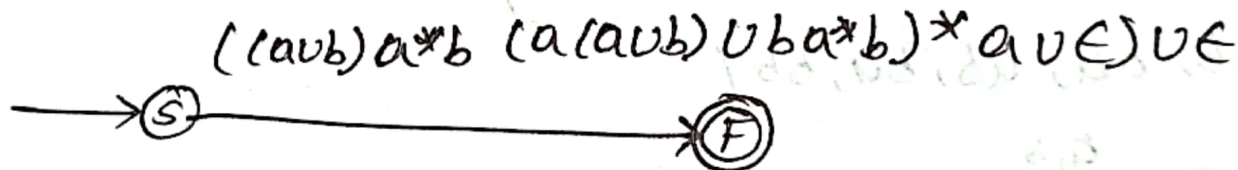
Step-2



Step-3



Step-4

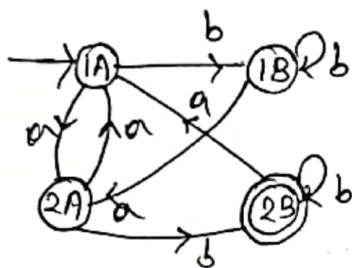


The regular expression, $((a \cup b) a^* b (a(a \cup b) \cup b a^* b)^* a \cup b) \cup \epsilon$.

Minimization of DFA

- ① Remove all states those are not reachable from initial state.
- ② Draw state transition
- ③ Mark initial and final state
- ④ 0 equivalent = (set of final state) (set of non final state)
- 1 equivalent *
- 2 equivalent -

Example: ① $L = \{w \mid w \text{ has an odd number of } a\text{'s and end with } ab\}$



Solution: Step-1 No change

Step-2

State	a	b
1A	2A 1B	
1B	2A 1B	
2A	1A 2B	
2B	1A 2B	

Step-3

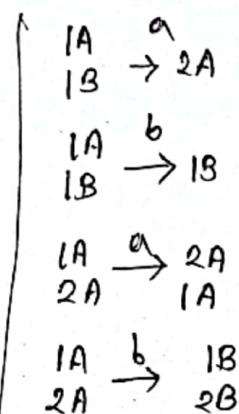
State	a	b
→ 1A	2A 1B	
1B	2A 1B	
2A	1A *2B	
*2B	1A *2B	

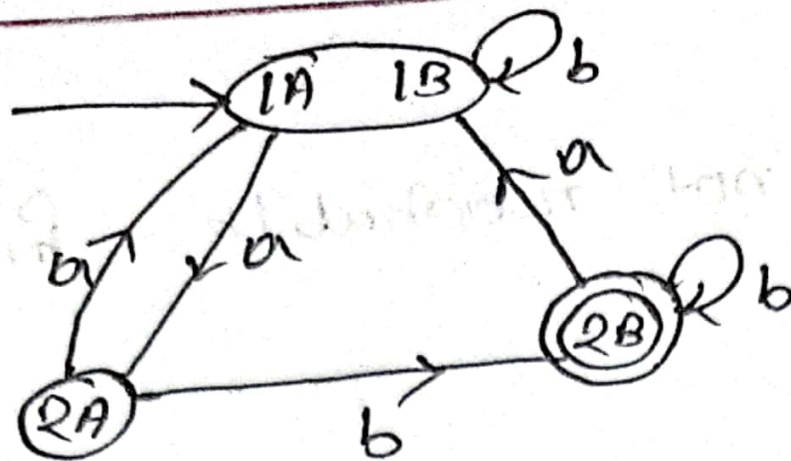
Step-4

0 equivalent - [2B] [1A 1B 2A]

1 equivalent - [2B] [1A 1B] [2A]

2 equivalent - [2B] [1A 1B] [2A]





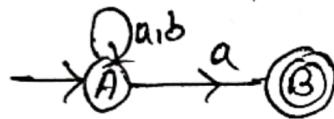
Minimization (of DFA)

Non deterministic finite automata (NFA): NFA contains 5 tuple,

$NFA \rightarrow (Q, \Sigma, \delta, q_0, F)$ where

- ① Q is a finite set of all states
- ② Σ is a finite alphabet set
- ③ $\delta: Q \times \Sigma \rightarrow P(Q)$ is the ~~start state~~ transition function
- ④ $q_0 \in Q$ is the start state
- ⑤ $F \subseteq Q$ is the set of final states

Example: $L = \{ \text{ends with } a \}$
 $= \{ a, aa, ba, aqa, \dots \}$



① $Q = \{A, B\}$

② $\Sigma = \{a, b\}$

③ $Q \times \Sigma \rightarrow 2^Q$

↓

$\{A, B\} \times \{a, b\}$

$\rightarrow (A, a), (A, b), (B, a), (B, b)$

$(A, a) \rightarrow \emptyset$

$(A, b) \rightarrow \{A\}$

$(B, a) \rightarrow \{B\}$

$(B, b) \rightarrow \{A, B\}$

Subset: $\emptyset$

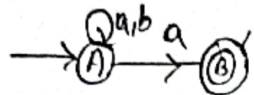
$\{A\}$

$\{B\}$

$\{A, B\}$

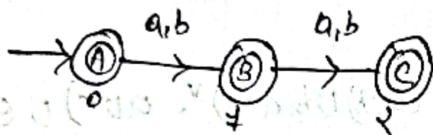
NFA Construction:

- ① $L = \{ \text{ends with } a \}$
 $= \{ a, aa, ba, aaa, \dots \}$



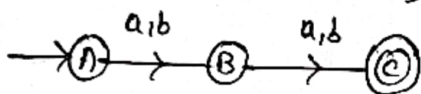
- ② $L = \{ a, b \}$

$L = \{ w \mid \text{the length of } w \text{ is at most } 2 \}$
 $= \{ \epsilon, a, b, aa, ab, ba, bb \}$



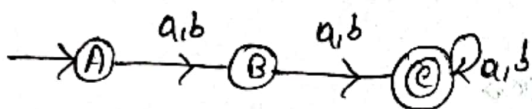
- ③ $L = \{ w \mid \text{the length of string exactly } 2 \}$

$L = \{ aa, ab, ba, bb \}$

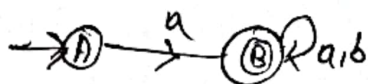


- ④ $L = \{ w \mid \text{the length of } w \text{ at least } 2 \}$

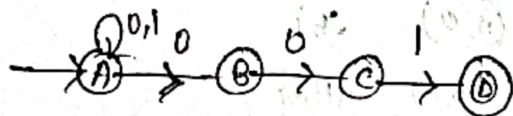
$= \{ aa, ab, ba, bb, aaa, \dots \}$



- ⑤ $L = \{ \text{starts with } a \}$

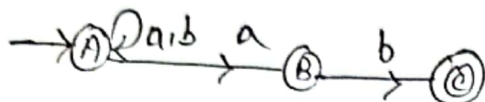


- ⑥ $L = \{ w \mid w \text{ ends with } 001 \}$



NFA to DFA

① $L = \{ \text{ends with } ab \}$



	a	b
→ A	{A, B}	{A}
B	∅	{C}
* C	∅	∅

S(NFA)

	a	b
→ [A]	[AB]	[A]
[AB]	[AB]	[AC]
* [AC]	[AB]	[A]

S(DFA)



DFA

② $L = \{ w \mid w \text{ starting with } a \}$
 $= \{ a, ab, aa, \dots \}$

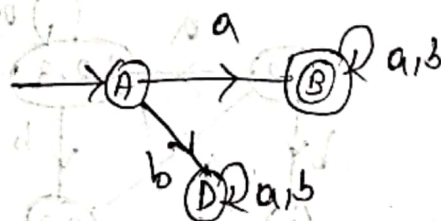


	a	b
→ A	{B}	{}
* B	{B}	{B}

S(NFA)

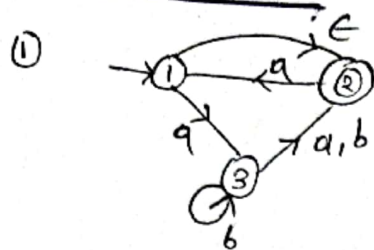
	a	b
→ [A]	[B]	[D]
→* [B]	[B]	[B]
[D]	[D]	[D]

S(DFA)



DFA

ϵ NFA to DFA:



Step-1

NFA transition

	a	b
1	{2}	∅
2	{1}	∅
3	{2}	{2, 3}

S(NFA)

Step-2

$$\epsilon \text{ closure}(1) = \{1, 2\}$$

$$\epsilon \text{ closure}(2) = \{2\}$$

$$\epsilon \text{ closure}(3) = \{3\}$$

Step-3

$$q_0' = \epsilon(q_0)$$

$$= \epsilon(1)$$

$$= \{1, 2\}$$

Step-4

DFA transition

State	a	b
$\rightarrow [1, 2]$	$[1, 2, 3]$	$[2]$
* $[1, 2, 3]$	$[1, 2, 3]$	$[2, 3]$
* $[2, 3]$	$[1, 2]$	$[2, 3]$
$[2]$	$[2]$	$[2]$

d	∅
[1] [2]	[1] [2]
[2] [3]	[2] [3]
[1] [2]	[1]

(1, 2, 3)

